Virtual Environments, Packages, Import/Export

Enrico Toffalini

Virtual Environment

– an **isolated Python environment** that allows you to install packages and manage dependencies **separately from your system/main Python installation**. This helps avoid conflicts between projects that may require different versions of the same packages. `venv`s are routinely used in professional projects. We will not use them systematically in this course, but know that they are **best practice for managing projects ensuring reproducibility**

BTW, something similar now exists also in R via the `renv` package, although unfortunately not widely used

Virtual Environment

Practically, it is a **local folder** with **an isolated full Python environment**. It contains:

- a copy of the **Python interpreter**;
- **all the packages you install** at the **exact versions** used at that time

This folder can ideally be placed inside your project directory

Virtual Environment

Create a virtual environment with this command in your bash/terminal:

```
python -m venv nameOfMyEnv
```

then activate it before using

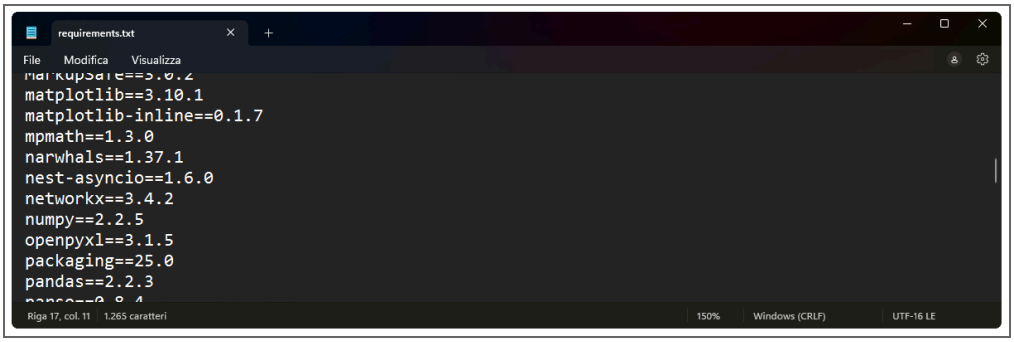
```
source nameOfMyEnv/bin/activate     # Lin

nameOfMyEnv\Scripts\activate.bat    # Win
# or alternatively
nameOfMyEnv\Scripts\activate.ps1    # Win
```

…alternatively, inside IDEs, you may activate the `venv` via specific commands like `reticulate::use_virtualenv("nameOfMyEnv", required=T)` (in ***RStudio***), or setting the Python interpreter manually and then restarting the kernel (in ***Spyder***)

Virtual Environment

Regardless of your main Python installation, you should **reinstall all packages needed for your project inside the local `venv`** (after activating it). This is considered best practice, as it ensures **isolation** across projects and **reproducibility**. At any time, you can export a **`requirements.txt`** file to document the exact versions of all installed packages (this is particularly useful for sharing your environment, e.g., via *GitHub*):

```
pip freeze > requirements.txt
```



Project like a pro 😎

- Use a **virtual environment** per project (as a subfolder);
- Place scripts, notebooks, and data in different subfolders;
- Use **relative paths** (e.g., `"data/myfile.csv"`);
- Save results and figures via code (not manually)

```
myProjectFolder/
├── venv/              ← virtual environme
├── data/              ← .csv, .xlsx, etc.
├── scripts/           ← .py scripts
├── results/           ← output files, fig
├── notebooks/         ← markdowns, colab
├── requirements.txt   ← list of installed
└── README.md          ← brief description
```

Installing and importing packages

Installing, inside an IDE console or Colab:

```
!pip install numpy  # install one package
!pip install numpy pandas seaborn  # insta
!pip install numpy==2.2.5  # install packa
```

Then, before using any of their functions, import packages and modules:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

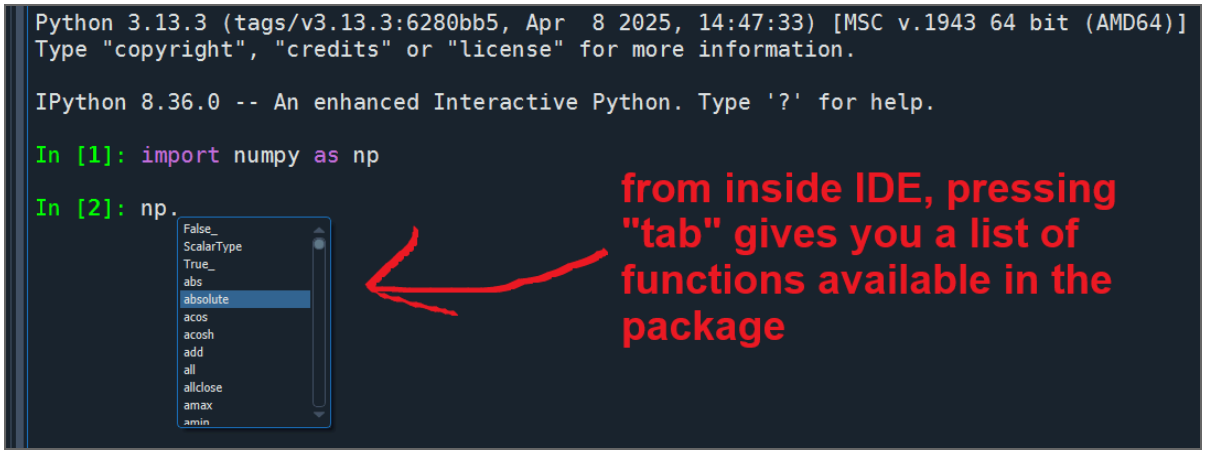
`as` gives a shorter ***alias*** to a package or module name (e.g., `pd` for `pandas`, `np` for `numpy`); this is convenient because in Python you frequently need to call different functions by *always* specifying the package/module name (unlike in R; unless you import individual functions, e.g., `from numpy import array`)

Using functions, help, autocomplete

Use a function from a package, and call **help**:

```
np.mean([2.0, 1.5, 7.1, 4.2]) # call a fun
?np.mean      # help only in IDE console o
help(np.mean) # help via built-in function
```

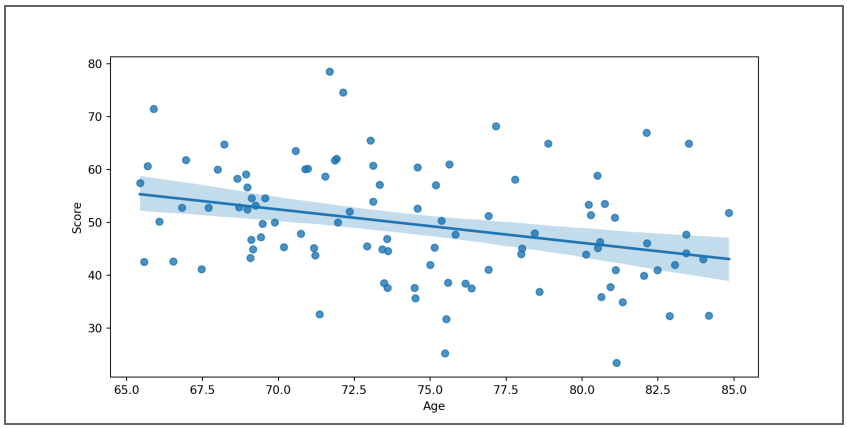
Use `tab` to **autocomplete and explore** available functions of a package ↴



Using functions, help, autocomplete

As in R, you can rely on positional order of arguments instead of naming them, or you can completely omit them if there are valid default arguments. However, it's best practice to make all relevant arguments explicit for readability and reproducibility

```
sns.regplot(data=myDF, x="Age", y='
plt.show()
```



Accessing functions as *methods*

In Python, objects may have functions *attached* to them: these are called **methods**, and are accessed using dot (`.`) notation *(more on this later!)*

```
import numpy as np
myVect = np.array([2.0, 1.5, 7.1, 4.2])
```

Method-style syntax , requires that the object has this method:

```
myVect.mean
```

```
np.float64(3.7)
```

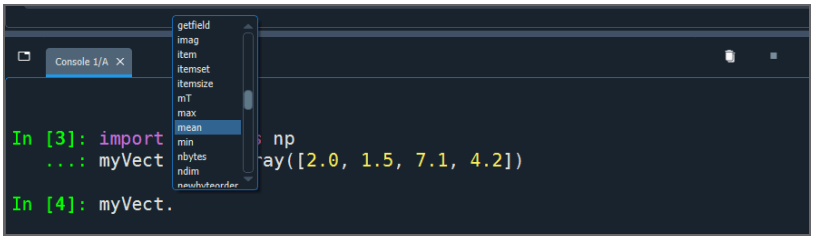
Function-style syntax (more like R):

```
np.mean(myV
```

```
np.float64(3.7)
```

Use `tab` to autocomplete and explore available

methods of an object →



Working directory

Equivalent to R's `getwd()` / `setwd()`:

```
import os

os.getcwd()                    # Get cu
os.chdir("path/to/folder") # Change
```

(in **Colab**, paths are relative to the notebook location in Google Drive)

Relative paths

```
path = "myProjectFolder/"
path = "../myProjectFolder/"
```

Absolute paths

```
path = "C:/Users/enric/Desktop/myPr
path = "/home/enric/Desktop/myProje
path = "/Users/enric/Desktop/myPro
```

Import/Export objects *(no equivalent of `save.image()` of R)*

```
import pickle

myObject = 10.5

with open("myObject.pkl", "wb") as file:
    pickle.dump(myObject, file)

with open("myObject.pkl", "rb") as file:
    myObject = pickle.load(file)
```

simpler version

```
myObject = 10.5

pickle.dump(myObject, open("myObject.pkl",

myObject = pickle.load(open("myObject.pkl"
```

(this simpler version is suboptimal because it doesn't properly close the file after using, but still works)

Import tabular data (more on pandas later!)

from CSV

```
import pandas as pd

df = pd.read_csv("data/Courses40Cycle.csv"
```

from Excel

```
df = pd.read_excel("data/Courses40Cycle.xl
```

from Ctrl+C copied elements (beautiful ❤️ but only for Windows)

```
df = pd.read_clipboard()
```

Export tabular data

```
df.to_csv("exported_data.csv", index=False

df.to_excel("exported_data.xlsx", index=Fa
```

Export figures

```
import matplotlib.pyplot as plt
plt.scatter(np.random.normal(0,1,10), np.r
plt.title("Example Plot")
plt.show()

plt.savefig("myPlot.png", dpi=300)  # expo
```

Delete an object *(similar to `rm(df)` in R)*

```
del df  # delete object named df
```

Listing all objects in workspace *(similar to `ls()` in R)*

```
dir()
```

```
['Age', 'N', 'Score', '__annotations__',
'__builtins__', '__doc__', '__loader__',
'__name__', '__package__', '__spec__', 'df',
'myVect', 'np', 'pd', 'pl', 'plt', 'r', 'sns']
```

dir()

`dir()` is a built-in function that does more than just returning a list of objects in workspace; it allows you to inspect all ***attributes*** and ***methods*** of any object

```
a = [5.2, 2, 0, 98, 11.5, 63.11]
dir(a)[37:47]
```

```
['append', 'clear', 'copy', 'count', 'extend',
'index', 'insert', 'pop', 'remove', 'reverse']
```

```
a = np.array([5.2, 2, 0, 98, 11.5, 63.11])
dir(a)[97:127]
```

```
['all', 'any', 'argmax', 'argmin',
'argpartition', 'argsort', 'astype', 'base',
'byteswap', 'choose', 'clip', 'compress',
'conj', 'conjugate', 'copy', 'ctypes',
'cumprod', 'cumsum', 'data', 'device',
'diagonal', 'dot', 'dtype', 'dump', 'dumps',
'fill', 'flags', 'flat', 'flatten', 'getfield']
```

```
import numpy as np
dir(np)[40:70]
```

```
['abs', 'absolute', 'acos', 'acosh', 'add',
'all', 'allclose', 'amax', 'amin', 'angle',
'any', 'append', 'apply_along_axis',
'apply_over_axes', 'arange', 'arccos',
'arccosh', 'arcsin', 'arcsinh', 'arctan',
'arctan2', 'arctanh', 'argmax', 'argmin',
```

```
'argpartition', 'argsort', 'argwhere', 'around',
'array', 'array2string']
```